

Matriz de seguridad — contenido HTML/JS en LMS/CMS educativos

Ernesto Serrano Collado · info@ernesto.es

2026-06-14

Índice

1. Tabla resumida (para el cuerpo del artículo)	1
2. Matriz técnica completa (anexo)	3
3. Mitigaciones emparejadas (riesgo → mitigación → limitación)	6
4. Tabla de concienciación (perfil no técnico)	6

Evidencia verificada contra el HEAD de cada repositorio (junio 2026). Cada celda “Cita” remite a `archivo:línea`. Las afirmaciones de comportamiento del navegador están marcadas [hecho] (verificado en código o prueba) o [inferencia].

SHAs de referencia: `mod_exelearning 2c5473d` · `mod_exeweb 60d24fb` · `mod_exescorm e985f4d` · `moodle 2104c372962` (5.x, code en public/) · `exelearning 8101f54e` · `wp-exelearning 9eb07ff` · `omeka-s-exelearning 33faf89`.

Origen de la evidencia. Las filas y atributos de versión **estable** / **legacy** corresponden a los SHAs fijados arriba; el **modo seguro** (atributos **secure** y los *toggles* de modo **iframe**, en las secciones 2.7 y 2.8) procede de la **propuesta de modificación de código** (prototipo), aún **no adoptada upstream**. El SHA estable de cada integración es *same-origin* (equivalente a **legacy**).

1. Tabla resumida (para el cuerpo del artículo)

Plataforma / recurso	¿Ejecuta JS del autor?	¿Mismo origen que el LMS?	sandbox del iframe	Aislamiento real
mod_page (Página)	Sí (verificado en ejecución: ejecuta <code><script></code>)	Sí	— (no es iframe)	Ninguno server-side (noclean); restringido por capacidad (RISK_XSS)
mod_scorm (core)	Sí	Sí	ninguno	Ninguno (confía en el SCO)
mod_h5pactivity / core_h5p	Parámetros No / librerías Sí (preloadedJs)	Sí (about:blank hereda origen)	— (<code>contentDocument.write</code> , sin sandbox)	Parámetros filtrados por semántica; el JS de librería se ejecuta <i>same-origin</i> , control de capacidad
mod_exelearning (estable)	Sí	Sí	allow-scripts allow-same-origin allow-popups allow-forms allow-popups-to-escape-sandbox	h5p:updatelibraries Parcial (mantiene allow-same-origin)
mod_exelearning (modo seguro)	Sí	No (opaco)	allow-scripts allow-popups allow-forms (sin allow-same-origin)	Fuerte (origen opaco + puente)
mod_exeweb	Sí	Sí	ninguno	Ninguno
mod_exescorm	Sí	Sí	ninguno	Ninguno
wp-exelearning (estable)	Sí	Sí	allow-scripts allow-same-origin allow-popups	Parcial (mantiene allow-same-origin)
wp-exelearning (modo seguro)	Sí	No (opaco)	allow-scripts allow-popups	Fuerte (origen opaco)
omeka-s-exelearning (estable)	Sí	Sí	allow-same-origin allow-scripts allow-popups allow-popups-to-escape-sandbox	Parcial (mantiene allow-same-origin)

Plataforma / recurso	¿Ejecuta JS del autor?	¿Mismo origen que el LMS?	sandbox del iframe	Aislamiento real
omeka-s-exelearning (modo seguro)	Sí	No (opaco)	allow-scripts allow-popups allow-popups-to-escape-sandbox	Fuerte (origen opaco; también vistas públicas)

Para cada integración mantenida se muestran **dos estados**: la **versión estable** (*same-origin*) y el **modo seguro propuesto** (origen opaco; propuesta de modificación de código). El modo **legacy** (*same-origin*) queda como respaldo **opcional** (véase la nota tras la tabla principal del artículo).

Lectura rápida: *ejecutar JavaScript del autor no es el problema; el problema es ejecutarlo con el mismo origen que el LMS y **sin** una frontera explícita*. Las dos columnas que de verdad importan son “mismo origen” y “aislamiento real”.

1.1 Matriz de riesgo formal

Criterio de clasificación explícito (referenciado desde la sección 3.4 del artículo): siete dimensiones observables más la separación entre **impacto máximo demostrado** (verificado en laboratorio) e **impacto inferido** (deducido del modelo del navegador). “SS” = *server-side*.

Mecanismo	JS autor	Mismo origen	Rol p/ publicar	Rol visitante	Token en DOM	CSP restr.	Mutación SS alcanzable (rol)	Impacto máx. demostrado	Impacto inferido
mod_page	Sí	Sí (top)	profesor/gestor (addinstance)	cualquiera	Sí (sesskey)	No	Sí (servicios AJAX)	autoedición del propio perfil (en ejecución)	acciones acotadas por el rol del visitante
mod_scorn	Sí	Sí	profesor	cualquiera	Sí	No	Sí (restringido por savetrack)	lectura de DOM/ <i>sesskey</i> (en ejecución)	forja acotada por validación SS
H5P · parámetros	No (filtrado)	Sí	—	cualquiera	n/a	No	n/a	control negativo (en ejecución)	—
H5P · librería	Sí (preloadedJs)	Sí	manager (updatelibraries)	cualquiera	Sí	No	Sí	ruta en código + PoC validada (manual)	JS arbitrario <i>same-origin</i>
mod_exelearning (estable)	Sí	Sí	profesor	cualquiera	Sí	No	Sí	lee <i>sesskey</i> y forja si el rol tiene capacidad mutadora (en ejecución)	escalado por rol
mod_exelearning (modo seguro)	Sí	No (opaco)	profesor	cualquiera	No (token solo-lectura)	Parcial (base; sin perfil estricto)	solo vía puente validado	SecurityError (en ejecución; Chromium y Firefox/Gecko, Playwright) — (solo código)	aislado
mod_exeweb / mod_exescorm	Sí	Sí	profesor	cualquiera	Sí	No	Sí	— (solo código)	acceso total <i>same-origin</i>
wp-exelearning / omeka-s-exelearning (estable)	Sí	Sí	autor/editor	cualquiera	Sí	No	Sí	acceso al padre / <i>/wp-admin/</i> (en ejecución)	escalado por rol
wp-exelearning / omeka-s-exelearning (modo seguro)	Sí	No (opaco)	autor/editor	cualquiera	No	Parcial (base; sin perfil estricto)	—	opaco (en ejecución; Chromium y Firefox/Gecko, Playwright)	aislado

CSP restr. = «Parcial (base; sin perfil estricto)»: el modo seguro preserva el origen opaco (directiva *sandbox* en la respuesta), pero *img-src/script-src/media-src* aún admiten *https*; el perfil de CSP estricta que cierra ese residual es opcional y queda como trabajo futuro (sección 6.3 del artículo).

2. Matriz técnica completa (anexo)

2.1 *mod_exelearning* (2c5473d)

Atributo	Valor	Cita
Cómo se publica	El profesor sube un .elpx; se extrae y se sirve por <code>pluginfile.php</code>	<code>lib.php:513-568</code>
HTML/JS arbitrario	Sí (HTML/JS del autor del paquete)	<code>view.php:446-447</code>
JS se ejecuta	Sí	<code>iframe allow-scripts</code>
Mismo origen	Sí (<code>\$CFG->wwwroot</code> único)	<code>view.php:446-447</code>
Usa <code>iframe</code>	Sí, <code>#exelearningobject</code>	<code>view.php:446-447</code>
<code>sandbox</code>	<code>allow-scripts allow-same-origin allow-popups allow-forms allow-popups-to-escape-sandbox</code>	<code>view.php:446-447</code>
<code>allow-top-navigation</code>	No (omitido a propósito)	<code>view.php:446-447</code>
<code>allow-popups-to-escape-sandbox</code>	Sí (residuo a quitar)	<code>view.php:446-447</code>
CSP	No emitida	<code>lib.php:513-568</code>
Permissions-Policy	No emitida (pendiente)	<code>lib.php:513-568</code>
X-Frame-Options	No (en <code>pluginfile</code>); core pone <code>sameorigin</code> en páginas	<code>weblib.php:1604-1605</code>
<code>postMessage</code>	Sí, solo en el editor (no en el visor)	<code>amd/src/editor_modal.js:441-449</code>
Valida <code>event.origin</code>	Sí (editor)	<code>editor_modal.js:441-449</code>
Valida <code>event.source</code>	Sí (<code>=== iframe.contentWindow</code>)	<code>editor_modal.js:441-449</code>
Token/contexto	<code>sesskey</code> validado server-side en <code>track.php</code>	<code>track.php:40, view.php:387-390</code>
Acceso a <code>parent</code>	Sí (mismo origen)	bridge SCORM
Acceso a cookies no-HttpOnly	Sí (mismo origen)	mismo origen
Acceso al <code>sesskey</code>	Sí (va en la URL de <code>track.php</code> , legible desde el DOM/JS)	<code>view.php:387-390</code>
Localiza formularios edición	Sí, si el DOM padre los tiene	mismo origen
Cambios reales	Sí (en ejecución): <code>rename</code> de <code>firstname</code> vía <code>core_user_update_users</code> forjado con el <code>sesskey</code> leído de <code>M.cfg</code> , sobre cuenta de laboratorio y revertido —con una sesión que posee <code>moodle/user:update</code> ; sin esa capacidad el servidor lo rechaza (frontera de rol)	<code>evidencias/resultados-modo-seguro.json; track.php:40</code>
Mitigaciones	<code>Sandbox</code> parcial; <code>require_sesskey</code> server-side <code>pluginfile</code> ; <code>require_sesskey</code> server-side	<code>lib.php:513-568, track.php:40</code>
Riesgos que quedan	XSS cross-component same-origin (riesgo residual)	—
Impacto de IA sin revisar	Alto: JS generado se ejecuta con el origen de Moodle	—

Dependencias **duras** de same-origin (impiden quitar `allow-same-origin` sin reescribir el bridge): 1. El padre lee `iframe.contentDocument` para mapear `objectid` → `js/scorm_tracker.js:75-86` y `:151-154`. 2. El hijo (pipwerks) recorre `window.parent` buscando `window.API` → `assets/scorm/SCORM_API_wrapper.js:62-118`. 3. El *teacher-mode hider* inyecta CSS en `contentDocument` → `classes/local/ui/teacher_mode_hider.php:44-61`.

2.2 *mod_exweb* (60d24fb) y *mod_exescorm* (e985f4d)

Atributo	<i>mod_exweb</i>	<i>mod_exescorm</i>
<code>iframe</code>	<code>embed_general.mustache:32-34</code>	<code>player.php:283-285 (noscript)</code>
<code>sandbox</code>	ninguno	ninguno
HTML/JS arbitrario	Sí	Sí
Mismo origen	Sí	Sí
SCORM API walk	No (sin seguimiento)	Sí (<code>SCORM_API_wrapper.js:71-78,140-142</code>)
<code>sesskey</code>	—	URL param (<code>datamodels/scorm_12.js:19-24</code>)
CSP en <code>pluginfile</code>	No (<code>lib.php:448-512</code>)	No (<code>lib.php:1064-1142</code>)
<i>teacher-mode hider</i>	Sí (<code>locallib.php:90-107</code>)	—

2.3 Moodle core: *mod_scorm* (2104c372962)

Atributo	Valor	Cita
<code>iframe</code> del SCO	<code>sin sandbox</code>	<code>public/mod/scorm/player.php:279-285</code>
API discovery	<code>myFindAPI</code> recorre <code>win.parent</code> y <code>window.opener</code>	<code>loadSCO.php:113-122, :128-129</code>

Atributo	Valor	Cita
sesskey	confirm_sesskey() antes de guardar	datamodel.php:53
Capability	mod/scorm:savetrack	datamodel.php:56
Mismo origen	Sí (wwwroot/pluginfile.php)	locallib.php:2323,2327

Veredicto: mod_scorm ejecuta HTML/JS no confiable **sin sandbox** — más débil que mod_exelearning. Su defensa es server-side (sesskey + capability), no aislamiento del cliente.

2.4 Moodle core: core_h5p / mod_h5pactivity (2104c372962)

Atributo	Valor	Cita
iframe	src="about:blank"	h5piframe.mustache:33
Construcción	contentDocument.write(...)	h5p.js:390-394
Origen del contenido	hereda el del padre (same-origin) [hecho]	h5p.js:391-394 (nota verificada)
HTML/JS arbitrario (parámetros)	No: content.json validado contra la semántica; filter_xss sin <script> ni on*	h5p.classes.php:2260-2282 (filterParameters), :4303-4384, :5033-5054
HTML/JS de librería (preloadedJs)	Sí: código de confianza, <script src=pluginfile.php/.../core_h5p> same-origin y sin sandbox	player.php:484-500, h5p.js:391-437
Gate de instalación de librería	moodle/h5p:updatelibraries (arquetipo manager, RISK_XSS), evaluada contra quien sube; profesorado solo h5p:deploy	lib/db/access.php, helper.php:210-224, api.php:403-405, h5p.classes.php:1577-1579
Whitelist	Extensiones curadas (content + librería)	framework.php:698-700
postMessage handler	Valida event.source===window.parent y event.data.context==='h5p'; no valida event.origin	embed.js:38-46, h5p.js:457-465
postMessage envío	targetOrigin = '*' (origen-agnóstico)	embed.js:64-73, h5p.js:484-493
\$CFG->h5pcrossorigin	CORS para media (img/audio/video), no para el iframe	helper.php:383, config-dist.php:778-786

Veredicto (matizado): hay que distinguir dos planos. En los **parámetros** de contenido, H5P **no ejecuta HTML/JS de autor**: filtra el texto con filter_xss contra una lista cerrada de etiquetas sin <script> y descarta on* (h5p.classes.php:4303-4384, :5033-5054) — control negativo, más controlado por diseño que un .elpx. En el plano de las **librerías**, en cambio, sí: el preloadedJs de una librería es **código de confianza** que se ejecuta *same-origin* y sin sandbox (player.php:484-500, h5p.js:391-437); la barrera para que el autor introduzca su propia librería es la **capacidad** moodle/h5p:updatelibraries (gestión/administración, RISK_XSS), no el saneamiento — **mismo patrón que mod_page**. PoC: poc/evil-h5p-library.h5p; evidencia: evidencias/resultados-h5p-library.json. Además **no es inmune** por contenido: XSS documentados (MDL-67110, CVE-2024-43439 —XSS reflejado vía mensaje de error—, CVE-2024-3111 —stored XSS vía SVG—) y precedente de RCE por import de .h5p (GHSA-mj4f-8fw2-hrfm / CVE-2026-30875, Chamilo). Su postMessage no valida event.origin (mitiga con event.source + context), patrón a no copiar a ciegas.

Acción privilegiada (Moodle, verificado en código): el contenido same-origin (Página, .elpx, SCO) lee window.M.cfg.sesskey y puede invocar servicios web **AJAX** como core_user_update_users ('ajax' => true, cap moodle/user:update, public/lib/db/services.php:1982-1989) → si lo abre un usuario privilegiado, puede forjar la edición de una cuenta. La defensa es server-side (capacidad + validación), no ocultar el token.

2.5 Moodle core: mod_page + filtrado global (2104c372962)

Atributo	Valor	Cita
Render	format_text(..., noclean=true)	public/mod/page/view.php:90-93
trusttext por defecto	0 (desactivado)	locallib.php:53
¿trusttext activo?	!empty(\$CFG->enabletrusttext)	weblib.php:958-961
¿texto confiable?	activo y capability	weblib.php:949-950
Capability	moodle/site:trustcontent (RISK_XSS, solo profesor/manager)	db/access.php:452-454
noclean en salida	\$formatoptions->noclean = true	public/mod/page/lib.php:352
purify_html (otros contextos)	HTMLPurifier quita <script> en texto no noclean	weblib.php:1105-1107
enabletrusttext por defecto	0 (desactivado)	public/admin/settings/security.php:68
X-Frame-Options	sameorigin salvo \$CFG->allowframembedding	weblib.php:1604-1605
CSP global	ninguna por defecto	no localizada en el flujo revisado de weblib.php

Veredicto (verificado en ejecución): en `mod_page`, `<script>` y `<img onerror>` se ejecutan — `noclean=true` hace que `format_text` traduzca a `clean=false` y **no** pase por `HTMLPurifier`. El filtrado `purify_html` se aplica a otros campos de texto no confiable (sin `noclean`), **no** a la salida de `mod_page`. La protección de `mod_page` es la **capacidad** de crear/editar la Página (`mod/page:addinstance`; HTML confiable ligado a `moodle/site:trustcontent`, `RISK_XSS`), no el saneado. `enabletrusttext=0` por defecto **no** impide la ejecución porque `mod_page` usa `noclean`.

2.6 eXeLearning core / contenido exportado (8101f54e)

Atributo	Valor	Cita
SCORM API walk	<code>window.parent (500) + win.top.opener</code> , sin validación de origen	<code>SCORM_API_wrapper.js</code> :71-77, :136-141
localStorage	clave <code>exeTeacherMode</code> (revela contenido de profesor)	<code>exe_export.js</code> :100-130
postMessage listener <code>eval()</code>	valida type, no <code>event.origin</code> sí (callbacks de carga de script, tooltips, onclick)	<code>asset_url_resolver.js</code> :905-906 <code>common.js</code> :805,810,757; <code>common_edition.js</code> :39
MathJax	local empaquetado (no CDN)	<code>common.js</code> :26-115
iframes exportados	sin sandbox	<code>asset_url_resolver.js</code> :933-936
Validación server-side	ninguna (HTML exportado es estático/self-contained)	—

2.7 wp-exelearning — estable (9eb07ff) y modo seguro propuesto

Origen. El comportamiento estable en 9eb07ff es *same-origin* (fila legacy); las filas `secure` y el ajuste `exelearning_iframe_sandbox_mode` son la **propuesta de modificación de código** (prototipo), aún no adoptada *upstream*.

Atributo	Valor	Cita	Origen
Modo iframe	ajuste <code>exelearning_iframe_sandbox_mode</code> : secure (def.) / legacy; helper único, <i>fail-safe</i> a <code>secure</code>	<code>includes/class-iframe-sandbox.php</code>	prototipo
iframe (secure)	<code>sandbox="allow-scripts allow-popups"</code> (sin <code>same-origin</code> → opaco) + <code>referrerpolicy="no-referrer"</code>	<code>public/class-shortcodes.php</code> (<code>sandbox_tokens()</code>)	prototipo
iframe (legacy)	<code>sandbox="allow-scripts allow-same-origin allow-popups"</code> (<code>same-origin</code>)	<code>class-iframe-sandbox.php</code> (<code>TOKENS_LEGACY</code>)	estable (legacy)
Teacher mode (secure)	server-side por la <code>src</code> (<code>exe-teacher/exe-teacher-toggler</code>), aplicado por el proxy de contenido	<code>includes/class-content-proxy.php</code>	prototipo
Content proxy	<code>permission_callback => '_return_true'</code> (lectura no autenticada por hash SHA1)	<code>includes/class-exelearning-rest-api.php</code> :44-50	estable
Nonce en guardado	usa <code>permission_callback</code> (capability), no <code>wp_verify_nonce</code>	<code>rest-api.php</code> :223	estable
Nonce en editor	<code>wp_verify_nonce</code> al cargar página	<code>class-exelearning-editor.php</code> :111	estable

2.8 omeka-s-exelearning — estable (33faf89) y modo seguro propuesto

Origen. El comportamiento estable en 33faf89 es *same-origin* (fila legacy); las filas `secure` y el ajuste `exelearning_iframe_mode` son la **propuesta de modificación de código** (prototipo), aún no adoptada *upstream*.

Atributo	Valor	Cita	Origen
Modo iframe	ajuste <code>exelearning_iframe_mode</code> : secure (def.) / legacy; helper único, <i>fail-safe</i> a <code>secure</code>	<code>src/Service/IframeSandbox.php</code>	prototipo
iframe (secure)	<code>sandbox="allow-scripts allow-popups allow-popups-to-escape-sandbox"</code> (sin <code>same-origin</code> → opaco); también en las dos vistas públicas (antes fijadas a <code>allow-same-origin</code>)	<code>ExeLearningRenderer.php</code> , <code>view/.../public/*-show.phtml</code>	prototipo

Atributo	Valor	Cita	Origen
iframe (legacy) src	añade <code>allow-same-origin</code> fijado por JS desde <code>window.location</code> (prefijo de base)	<code>IframeSandbox::tokens()</code> <code>ExeLearningRenderer.php</code>	estable (legacy) estable
CSRF	token obligatorio (rechaza vacío/nulo)	<code>src/Controller/CsrfValidationTrait.php</code> 14-38	estable
postMessage	mixto: <code>editor.js</code> :83 usa <code>'*'</code> ; <code>omeka-exe-download.js</code> :122-125 y <code>omeka-exe-bridge.js</code> :46 usan origen específico	citadas	estable

3. Mitigaciones emparejadas (riesgo → mitigación → limitación)

Riesgo	Mitigación	Qué no resuelve
Lectura de cookies de sesión	<code>HttpOnly</code> + <code>SameSite=Strict</code>	No protege frente a XSS en el mismo origen ni a tokens expuestos en el DOM
Acceso al <code>parent</code> / mismo origen	iframe de origen opaco sin <code>allow-same-origin</code> (o <code>srcdoc</code> , o subdominio)	Rompe el bridge SCORM directo (<code>window.API</code>); exige bridge <code>postMessage</code>
Localización/envío de formularios con <code>sesskey</code> /nonce <code>postMessage</code> no validado	Validación server-side del token por acción + capacidades mínimas Lista blanca cerrada + validación de <code>event.origin</code> y <code>event.source</code>	Si el server confía en el cliente, ocultar el token en el DOM no sirve Una lista blanca mal mantenida reabre la superficie
Ejecución de JS arbitrario	CSP restrictiva + <code>sandbox</code> sin <code>allow-same-origin</code>	El contenido legítimo que necesita JS (SCORM/H5P) exige arquitectura de bridge explícita
Navegación superior / popups	Omitir <code>allow-top-navigation</code> y <code>allow-popups-to-escape-sandbox</code>	Puede degradar UX de contenido legítimo que abre recursos externos

Principio rector: la mitigación efectiva vive en el **servidor** y en las **cabeceras**, no en confiar en que el cliente “no encuentre” el token.

4. Tabla de concienciación (perfil no técnico)

Riesgo detectado	Por qué importa	Mitigación	Limitación de la mitigación
El recurso puede leer la cookie de sesión	Permitiría suplantar al usuario autenticado	<code>HttpOnly</code> + <code>SameSite=Strict</code>	No cubre XSS en el mismo origen
El recurso accede al marco padre	Acceso al DOM autenticado del LMS	Origen opaco sin <code>allow-same-origin</code>	Rompe SCORM directo; exige bridge
El recurso localiza el <code>sesskey</code> /nonce	Primer paso para forzar acciones	Validación server-side por acción	Inútil si el server confía en el cliente
El recurso localiza formularios de edición	Permitiría crear/modificar contenido	Capacidades mínimas + validación server-side	Depende de la configuración de roles
<code>postMessage</code> sin validar origen/source	Canal de control no autenticado	Allowlist + validación <code>origin/source</code>	Allowlist mal mantenida reabre el hueco